

Network Intrusion Detection

A Build-First Learning Framework

Decision Tree & SVM classifiers on CICIDS 2017 — structured as self-paced sprints, not a calendar.

Chineme Dimkpa · M.S. Software Engineering, UTA · Application Security → AI Security

How this document works

This is a **framework, not a script**. It locks down the things that are actually stable — your principles, the sprint structure, the data→tree→SVM arc, and a fully detailed Sprint 0 and Sprint 1. Later sprints are sketched as *direction*, not dictated task-by-task, because what you do in Sprint 5 depends on what broke in Sprint 2. You re-plan at each checkpoint with what you actually learned.

No dates. Sprints advance when the **exit check** is met — not when a day on the calendar arrives. A sprint might take you 3 days or 3 weeks. Both are fine.

The Principles (the part you don't get to renegotiate mid-project)

These exist because the convenient path and the learning path diverge, and they diverge most exactly when you're tired or behind. Written down now, while you're clear-headed, so they can outvote you later.

- **You write the code.** For this project, AI is the partner you ask “why does this break” and “what should I research” — never the thing that produces the file. The friction is the curriculum, not an obstacle to it.
- **Tight loop with the material, not with the AI.** You write a few lines, they break, you investigate, you fix, the fix teaches you. If the AI is the one watching code break and fixing it, the learning is happening to the wrong party.
- **Theory is pulled in just-in-time.** You don't read a chapter then look for where it applies. You hit a concrete confusion in the data or the model, *then* read the slice of AIMA that answers it.
- **Passive exposure is a first pass, never the only pass.** Podcasts and summaries build vocabulary. They are not study. The test of knowing is whether you can explain it cold, without looking.
- **Push back is the service.** When your reasoning has a gap, the useful move is to name it — not to affirm. Affirmation feels good and teaches nothing.
- **“Done” is defined, not felt.** Every sprint has an exit check. You advance because you passed it, not because you got bored.

Anatomy of Every Sprint

Whatever the sprint is about, it has the same five parts. This is the container; the content changes.

Part	What it is
1. Build task	A concrete thing you make by hand. The sprint is anchored here. Everything else serves it.
2. The struggle you'll hit	The friction predicted in advance — so when it happens you recognize it as the lesson, not a
3. Just-in-time theory	The specific slice of AIMA / Coursera to read once the struggle makes the question real.
4. Questions to bring here	What to surface in this chat — concepts to pressure-test, reasoning to check.
5. Exit check	The concrete proof the sprint landed. Pass it to advance. No pass, no advance.

Documentation Discipline (runs across all sprints)

- **LEARNING_LOG.md** — what you did each session, hours, what you read. The audit trail.
- **THOUGHTS.md** — raw thinking, your challenges to the book, insights as they land. Where understanding gets forged.
- **QUESTIONS.md** — running list, each tagged **[Discussed]**, **[Book]**, or **[Unsolved]**. An **[Unsolved]** that survives two sprints is a flag to bring here.

The Arc — Sprint Map

Detailed where it's honest to be (0 and 1), directional after. You and this chat re-plan the next sprint in detail at each checkpoint.

Sprint	Focus	Status
0	Setup & first contact with the data	Fully specified
1	Understand the data — EDA & the mess	Fully specified
2	Clean & prep — make it model-ready	Directional
3	Decision Tree — first working classifier	Directional
4	Decision Tree — evaluate & understand	Directional
5	SVM — gated on DT landing cleanly	Stretch / directional
6	Compare, write up, portfolio repo	Directional

Note the asymmetry between Sprints 3–4: building the tree is one sprint, understanding it is a whole separate one. That's deliberate. A model that runs is not a model you understand, and in security the gap between those two is where you get burned.

Sprint 0 — Setup & First Contact

Goal: a working environment and your first honest look at the raw data. Small on purpose. The point is to start the loop, not to achieve anything impressive.

Build task (you, by hand)

- Create the repo and the three markdown docs. Make the first LEARNING_LOG entry today.
- Stand up the environment: a virtual env, then pandas / numpy / matplotlib / seaborn / scikit-learn / jupyter. Confirm a notebook actually runs.
- Download CICIDS 2017. Load *one* CSV into a dataframe. Don't clean it. Don't model it. Just load it.
- Look at it: `.shape`, `.columns`, `.head()`, `.info()`, and the value counts of the label column.

The struggle you'll hit (this is the curriculum)

- The files are big enough to make you think about memory for the first time.
- Column names have quirks — leading spaces are a known CICIDS gotcha that will break selections in confusing ways.
- The label distribution is wildly lopsided. Seeing that number is the first real lesson, not a footnote.
- Dependency / environment friction. Annoying, universal, worth learning to resolve yourself.

Just-in-time theory

Almost none yet — this sprint is deliberately pre-theory. The only reading: skim the CICIDS 2017 documentation so you know what the capture actually *is* (which days, which attacks). Your Huawei network-analysis background is the asset here; lean on it.

Questions to bring to this chat

- “Here's my label distribution — what does this imbalance mean for everything downstream?”
- “Should I work on one day's file or combine them? What's the tradeoff?”

Exit check — Sprint 0

You can state, from memory: the shape of your data, what the label column contains, and the rough class balance. The repo exists, the three docs exist, a notebook runs. You have written down — in your own words, in THOUGHTS.md — one thing about the raw data that surprised you.

Sprint 1 — Understand the Data (EDA & the Mess)

Goal: go from “I loaded it” to “I understand what I'm holding and what's wrong with it.” You are **not** cleaning yet — you're diagnosing. Naming the disease before reaching for medicine.

Build task (you, by hand)

- Systematically profile the features: types, ranges, how many there are, which look continuous vs. categorical.
- Hunt the known pathologies yourself: NaNs, infinities, duplicate rows, columns that are constant or near-constant.
- Visualize. Distributions of a few features. The class imbalance as an actual chart. Begin a feel for which features might separate attack from benign — this connects straight to your prior insight that *features determine where data lands*.
- Write a short data-quality summary in THOUGHTS.md: what's broken, how much, and your *hypotheses* about why.

The struggle you'll hit

- Deciding what's signal vs. noise with no labels-of-correctness to tell you. This ambiguity is real ML work, not a gap in the tutorial.
- The imbalance stops being a number and becomes a problem: if 80%+ is benign, what does “95% accurate” even mean? (Hold that thought — it detonates in Sprint 4.)
- Infinities and NaNs force a real decision — and you'll want to resist fixing them now. Diagnose first.

Just-in-time theory

- AIMA on what a feature / attribute *is* and feature space — now that you've seen 80 columns, it'll mean something.
- Coursera: the EDA / pandas-manipulation sections, pulled in exactly as you need the operations.
- Concept to read with intent, not in the abstract: **why accuracy is a misleading metric under class imbalance**. You don't need the full evaluation theory yet — just enough to know the trap is coming.

Questions to bring to this chat

- “Here are the features I think matter and why — where's my reasoning weak?”
- “I found infinities in these columns. What are the real options, and what does each cost me?”
- “How do I think about which features to keep without leaking the answer into the model?”

Exit check — Sprint 1

You can explain, cold: how many features you have and their broad types, every data-quality problem you found and its rough scale, and a *defensible* first guess at which features carry signal. You can articulate why accuracy alone will mislead you on this dataset — before you've trained anything. THOUGHTS.md holds your written data-quality diagnosis.

Sprints 2–6 — Direction, Not Dictation

Sketched so you see the arc. Each gets fully specified in this chat at its checkpoint, using what actually happened in the sprint before. If I detailed them now I'd be writing fiction — and rebuilding the waterfall this framework exists to avoid.

Sprint 2 — Clean & Prep

Turn diagnosis into action. Handle the infinities/NaNs you found, decide on duplicates, encode the label, think about train/test splitting *before* you touch a model — and confront the imbalance question directly (resample? class weights? leave it and measure honestly?). Every choice here is a decision with consequences you'll have to defend, not a default to copy.

Exit check (provisional): A clean, split, model-ready dataset, and a written justification in THOUGHTS.md for every cleaning decision — including the ones you chose *not* to make.

Sprint 3 — Decision Tree — First Working Classifier

Build the simplest honest Decision Tree on your prepped data. Goal is a model that runs and produces predictions — not a good one yet. This is where your insight that *decision trees are explainable rational thinking* meets reality: you'll be able to actually look at the splits. Read AIMA on entropy and information gain *now* — the tree in front of you makes the math concrete.

Exit check (provisional): A trained tree that predicts on held-out data, and you can explain in plain words how it makes a single decision by tracing one path from root to leaf.

Sprint 4 — Decision Tree — Evaluate & Understand

The real intellectual core. Confusion matrix, precision, recall, F1 — and here the imbalance trap from Sprint 1 detonates. Confront *false negatives* vs. *false positives* in a security frame, where a missed intrusion costs more than a false alarm. Meet overfitting as the deliberate tradeoff you named, not a bug. This sprint is the gate: SVM only begins once the tree is genuinely understood, not merely running.

Exit check (provisional): You can read your confusion matrix in security terms, explain why your chosen metric fits an IDS, and show evidence about whether your tree generalizes or memorized. **Clearing this is what unlocks Sprint 5.**

Sprint 5 — SVM — The Comparison (gated)

Only if Sprint 4 landed cleanly. Same prepped data, different model family. The learning is in the *contrast*: SVM's decision boundary and margins vs. the tree's splits, where feature scaling suddenly matters (it didn't for the tree), the explainability you trade away. This is where “features determine where data lands” gets geometric.

Exit check (provisional): A trained SVM evaluated on the same footing as the tree, and you can explain why each model wins where it wins — in terms of how each draws boundaries, not just which scored higher.

Sprint 6 — Compare, Write Up, Portfolio Repo

Synthesis. Honest head-to-head, the writeup that makes your reasoning legible to someone else, and a repo a hiring manager — or your Fall 2026 AI Fundamentals self — can actually read. The deliverable

isn't the models; it's the documented thinking around them.

Exit check (provisional): A clean public repo: clear README, reproducible notebooks, an honest comparison, and the LEARNING_LOG / THOUGHTS / QUESTIONS docs telling the story of how you got there.

The checkpoint ritual (do this between every sprint)

Before starting the next sprint, come here with three things: (1) what the exit check actually produced, (2) what surprised you or broke, (3) what's now in QUESTIONS.md as [Unsolved]. We use that — not this document — to specify the next sprint in detail. The framework holds; the plan stays loosely held. That was the whole point.